

```

import numpy as np
import pandas as pd
from sklearn.datasets import load_iris
# loading iris data
iris_features, iris_label = load_iris(return_X_y=True)

# %%
# Check point 1 split validation set
from sklearn.model_selection import train_test_split

X, X_test, y, y_test = train_test_split(
    iris_features, iris_label, test_size=0.2, random_state=2
)
X_train, X_validate, y_train, y_validate = train_test_split(
    iris_features, iris_label, test_size=0.15, random_state=2
)

# %%
# Creating pipeline
from sklearn.decomposition import PCA
from sklearn.linear_model import LogisticRegression
from sklearn.pipeline import Pipeline
from sklearn.metrics import accuracy_score

def pipeline_ncomp(n):
    steps = [("Dim", PCA(n_components=n)), ("classify",
LogisticRegression())]
    return Pipeline(steps)

for n in range(1, 5):
    pipe = pipeline_ncomp(n)
    pipe.fit(X_train, y_train)
    y_pred = pipe.predict(X_validate)
    print(f"Component {n}")
    print("%.2f" % accuracy_score(y_validate, y_pred))

```

```
# %%
# Checkpoint 4 try to use cross validation

from sklearn.model_selection import StratifiedShuffleSplit
from sklearn.metrics import accuracy_score

sss1 = StratifiedShuffleSplit(n_splits=5, test_size=0.2, random_state=24)
sss1.get_n_splits(X, y)
print(sss1)
i = 1
for train_index, test_index in sss1.split(X, y):
    X_train, X_validate = X[train_index], X[test_index]
    y_train, y_validate = y[train_index], y[test_index]
    for n in range(1,5):
        pipe = pipeline_ncomp(n)
        pipe.fit(X_train,y_train)
        y_pred = pipe.predict(X_validate)
        print(f"Split {i}, Component {n}")
        print("%.2f" % accuracy_score(y_validate, y_pred))
    i = i+1
```

```
# %%
# try k fold cross validtaion
from sklearn.model_selection import KFold
i = 1
kfc = KFold(n_splits=5 )
for train_index, test_index in kfc.split(X):
    X_train, X_validate = X[train_index], X[test_index]
    y_train, y_validate = y[train_index], y[test_index]
    for n in range(1,5):
        pipe = pipeline_ncomp(n)
        pipe.fit(X_train,y_train)
        y_pred = pipe.predict(X_validate)
        print(f"Split {i}, Component {n}")
        print("%.2f" % accuracy_score(y_validate, y_pred))
    i = i+1
```

```
# %%
from sklearn.pipeline import Pipeline
from sklearn.model_selection import GridSearchCV
from sklearn.decomposition import PCA
from sklearn.linear_model import LogisticRegression
```

```
pipe = Pipeline([
    ('Dim', PCA()),
    ('classify', LogisticRegression())
])

param_grid = {
    'Dim__n_components': [1, 2, 3, 4]
}

grid = GridSearchCV(pipe, param_grid=param_grid, cv=5)
grid.fit(X_train, y_train)

print(grid.best_estimator_)
print(grid.best_params_)
print(grid.best_score_)

y_pred = grid.predict(X_test)

from sklearn.metrics import classification_report
print(classification_report(y_test, y_pred))
```